

Exact Policy-Space Credential Derivation for Legacy Password Rotation

Yuanyi Zhang

Independent Researcher

Abstract

Enterprise services often retain password-only authentication even when their users and endpoints are centrally managed. Deterministic password generators can avoid storing one password value per service, but commonly used encoders do not characterize the distribution over the complete set of policy-compliant passwords, and independent derivations may repeat during rotation. This paper presents a versioned system construction that compiles a bounded password policy into a finite deterministic automaton, counts its accepted language exactly with arbitrary-precision integers, and provides a canonical rank/unrank bijection. A credential generation is then mapped through a domain-separated finite-domain permutation before unranking. The construction is an application of established regular-language ranking and rank-then-encipher format-preserving encryption; we do not claim a new FPE, KDF, secret-sharing, or sampling primitive. Its system-level contribution is to bind this construction to explicit policy epochs, stable credential salts, record versions, history exclusion, and evidence-bounded remote rotation while preserving the previous derivation path for existing credentials. Under a total permutation abstraction, outputs comply with the policy and cannot repeat within one credential epoch before domain exhaustion; in an ideal permutation experiment, each fixed generation is uniform over the accepted language. A Rust prototype includes exact counting, rank/unrank, an FF1 cycle-walking research backend, fixed vectors, explicit v2-to-v3 migration, and an optional factor-preserving recovery profile. In a synthetic 12-policy corpus, 11 policies compile within a 250,000-state budget and one is rejected closed. On one host, an 18-character policy compiles in approximately 5–7 ms, and complete v3 derivation has a median of approximately 76 microseconds. A bounded TLA+ model explores 64 distinct lifecycle states without violating

six specified invariants. These results establish feasibility and expose state explosion and backend-totality limits; they do not establish universal policy compatibility or production network-recovery readiness.

Keywords: deterministic password generation, password policy, finite automata, format-preserving encryption, credential rotation, secret sharing

1. Introduction

Passwordless authentication is preferable when a service can be changed, but many internal Web applications, appliances, and vendor systems expose only a password interface. A client-side password manager must then retain or regenerate service credentials. Deterministic generators such as PwdHash, PALPAS, AutoPass, Spectre, and MFDPG investigate variants of this design space [1, 2, 3, 4, 5]. They reduce stored password values, but the generator must also satisfy service-specific composition rules and survive changes in the root key, policy, account identity, and remote password state.

Two questions are easily blurred. First, does every random choice made by an encoder avoid modulo bias? Second, is the *final* output uniform over all strings accepted by the complete policy? The first property does not imply the second. For example, a generator that first inserts one required character per class and then shuffles can assign different masses to legal strings with different class multiplicities. A second issue appears during rotation: a deterministic function evaluated independently at generations $g_1 \neq g_2$ can return the same password. The probability may be small in a large domain, but there is no structural non-repetition guarantee.

Established work already supplies the mathematical tools needed to address these issues. Accepted words of a finite regular language can be counted and ranked, and rank-then-encipher maps arbitrary finite formats through a keyed permutation [6, 7, 8]. Our research question is therefore not whether these tools can be reinvented for passwords. It is whether an executable, versioned credential lifecycle can use them without silently changing existing credentials, while giving precise security and failure boundaries.

This paper makes three scoped contributions:

1. It defines a canonical, resource-bounded policy IR and implements exact arbitrary-precision counting plus a shortlex rank/unrank bijection for the supported policy subset. Unsupported state growth fails closed.

2. It instantiates generation-indexed rank-then-encipher as derivation v3 and binds its key and tweak to credential identity, Root-Key generation, policy hash, and policy epoch. Credential salt remains stable inside the credential lineage. Existing v2 records retain byte-for-byte dispatch, and migration is an explicit pending remote rotation.
3. It integrates bounded history exclusion and evidence-based reconciliation with the derivation sequence, and evaluates distribution, collision, compiler growth, local performance, and lifecycle invariants. The artifact includes a fixed vector and a bounded TLA+ model.

The paper intentionally avoids stronger claims. The construction is not a new FPE or regular-language algorithm. The synthetic policy corpus is not a compatibility study. The bounded FF1 cycle-walking backend is a research instance that can fail closed after a configured number of walks. The optional peer-recovery implementation assumes honest-but-curious OPRF nodes and is not a production recovery service.

2. Related work and novelty boundary

2.1. *Deterministic password generation*

PwdHash derives site-specific passwords in the browser but does not address a versioned enterprise rotation lifecycle [1]. PALPAS combines a high-entropy master secret with per-site salts and synchronizes non-secret parameters [2]. AutoPass studies automatic generation and changing of site passwords [3]. Spectre specifies deterministic, site-scoped password templates [4]. These systems establish that deterministic service isolation and password templates are not new contributions.

MFDPG is the closest password-policy baseline. It surveys deterministic generators, converts regular-expression policies through NFA and DFA representations, and drives an Xeger/RandExp-style generator with a deterministic HMAC-DRBG [5]. Its artifact selects among available regular-expression branches using its generator rather than weighting every edge by the number of accepted completions. Our distinction is not “using a DFA.” It is the exact accepted-space model and its binding to a non-repeating, generation-indexed credential lifecycle. This distinction is narrower than proposing a new password-generation primitive.

2.2. Counting, ranking, and finite-domain encryption

Ranking finite-language strings and uniform generation from regular languages have a long history [6, 9]. Black and Rogaway formalized ciphers on arbitrary finite domains [7]. Bellare et al. explicitly described rank-then-encipher FPE and applied ranking and unranking to regular languages [8]. Consequently, the composition “regular language + rank/unrank + keyed permutation” is prior art. Our claims begin at the credential record and lifecycle binding, including stable salt semantics, separate root and policy generations, history-window rederivation, failure-closed dispatch, and remote evidence.

NIST SP 800-38G standardizes FF1 and FF3-family format-preserving encryption; the 2025 revision draft removes FF3 and requires an FF1 domain of at least one million [10]. Small domains also require care against message-recovery and codebook-style attacks [11]. We use FF1 only as a replaceable prototype backend and enforce the draft’s minimum domain. The security statements are parameterized by a finite-domain permutation interface rather than equating the interface with this bounded implementation.

2.3. Key recovery and multifactor derivation

MFKDF derives one key from threshold combinations of authentication factors and supports factor recovery [12]. Subsequent analysis identified state-integrity and specification concerns in the original construction [13]. Our system instead generates a random 256-bit Root Key and uses standard Shamir 2-of-3 sharing [14]. This is a conventional recovery choice, not an algebraic contribution. Share-set identity, generation, manifest KCV, and authenticated envelopes prevent mixing lifecycle states.

3. System and threat model

The client serves legacy applications that accept a username and password and need not be modified. A random Root Key K_{root} is created locally. The client stores credential description records (CDRs), not service-password values. A CDR contains the service and account identifiers, a 128-bit credential salt, Root-Key generation, credential generation, policy identifier and version, canonical encoding descriptor, derivation and encoder versions, policy epoch for v3, and a MAC under a Root-Key-derived record key.

We assume standard confidentiality and pseudorandomness properties for HKDF, HMAC, authenticated encryption, and the chosen permutation.

The endpoint is trusted while deriving or submitting a password. Endpoint compromise while the Root Key is unlocked can disclose generated passwords and is outside the construction’s protection. Service compromise reveals that service’s current password and policy but should not reveal the Root Key or directly compute other services’ credentials. The remote service is not a transaction participant: update responses may be lost, authentication replicas may lag, and old and new passwords may both work temporarily.

The adversary may copy, roll back, omit, or mix local records and recovery objects. MACs detect unauthorized modification once the correct Root Key is available; an independent freshness anchor is needed to distinguish a valid old snapshot from the newest committed snapshot. Availability, online guessing at the legacy service, server-side password hashing quality, and malicious password-policy changes are not solved by deterministic derivation.

4. Policy model and exact bijection

4.1. Supported policy IR

A policy P defines a finite language $\mathcal{L}_P \subseteq \Sigma^*$ with bounded length $L_{min} \leq |w| \leq L_{max}$. The version-1 policy IR supports an ordered effective alphabet, forbidden characters, class minimum and maximum counts, fixed positions, fixed prefix and suffix, forbidden first and last characters, per-character maximum totals, maximum identical and sequential ASCII runs, and finite forbidden substrings. ‘null’, empty, and absent fields are distinguished by canonical JSON serialization. The alphabet order is part of the protocol.

The compiler constructs a reachable deterministic product state containing only data required by enabled constraints. Minimum counters saturate after the minimum is met; maximum counters do not. Prefix/suffix and fixed-position constraints prune transitions. A bounded suffix state recognizes forbidden substrings. The compiler rejects an empty language, inconsistent class, unsupported normalization, length above 128, or more than 250,000 reachable states. Thus “supported” means expressible by this IR *and* compilable within its explicit resource budget, not arbitrary PCRE or semantic policy code.

4.2. Exact counting

Let the compiled automaton be $A_P = (Q, \Sigma, \delta, q_0, F, L_{min}, L_{max})$. For a target length ℓ , define

$$C_\ell(i, q) = |\{x \in \Sigma^{\ell-i} : \delta^*(q, x) \in F\}|. \quad (1)$$

The boundary and recurrence are

$$C_\ell(\ell, q) = \begin{cases} 1 & q \in F \\ 0 & q \notin F, \end{cases} \quad (2)$$

$$C_\ell(i, q) = \sum_{c \in \Sigma: \delta(q, c) \neq \perp} C_\ell(i+1, \delta(q, c)). \quad (3)$$

The exact space is $N_P = \sum_{\ell=L_{min}}^{L_{max}} C_\ell(0, q_0)$. Every count is an arbitrary-precision non-negative integer. ‘entropyBits’ reports $\log_2 N_P$ as the entropy of a uniform random variable over the language; a fixed keyed derivation output is deterministic and must not itself be described as having that entropy.

4.3. Rank and unrank

Strings use shortlex order: length first, then the canonical alphabet order. To unrank $r \in [0, N_P)$, the algorithm subtracts complete length blocks, then examines each outgoing character in order. The count on that edge is the size of its contiguous rank interval. Ranking performs the inverse additions.

Lemma 1 (bijection). If the count table is correct, $\text{Unrank}_P : [0, N_P) \leftrightarrow \mathcal{L}_P$ is a bijection and Rank_P is its inverse.

Proof sketch. At each state, the recurrence partitions the completion set into disjoint, adjacent intervals indexed by the next character. Their sizes sum to the current completion count. Induction on the remaining length gives existence and uniqueness; the outer shortlex length blocks are disjoint by construction. $\square$

5. Generation-indexed finite-domain derivation

The Root Key first yields a vault-scoped subkey. A 256-bit credential key is then derived with HKDF using the 128-bit credential salt and canonical context containing ‘serviceID’ and ‘accountID’:

$$K_{vault} = \text{HKDF}(K_{root}, \text{credential-permutation/v3}), \quad (4)$$

$$K_{cred} = \text{HKDF}(K_{vault}, \text{credentialSalt}, \text{Canonical}(\text{serviceID}, \text{accountID})). \quad (5)$$

The exact byte-level construction is specified by canonical JSON contexts and the artifact’s fixed vector. The permutation tweak binds the vault, service, account, credential salt, Root-Key generation, policy identity, policy hash, policy epoch, derivation version, and encoder version. Credential generation is deliberately excluded from the tweak because it is the permutation input.

Let $\Pi_{K,T}$ be a total keyed permutation on $[0, N_P)$. For generation $0 \leq g < N_P$,

$$\text{Derive}(K, T, g, P) = \text{Unrank}_P(\Pi_{K,T}(g)). \quad (6)$$

The prototype implements the interface using FF1-AES-256 over the smallest binary superset followed by cycle walking. It rejects domains below one million, domains above 512 bits, inputs outside the domain, and derivations that exceed 1,024 walks. This limit makes the backend a partial, failure-closed implementation. The following properties therefore refer to the abstract total permutation, or to successful calls of the prototype where stated.

Theorem 1 (compliance). Every successful derivation belongs to $\mathcal{L}_P$. This follows directly from Lemma 1.

Theorem 2 (epoch-local non-repetition). For fixed (K, T, P) and $g_1 \neq g_2$ in the domain, successful outputs differ. Both Π and Unrank are injective.

Theorem 3 (ideal-permutation marginal). If Π is sampled uniformly from all permutations on $[0, N_P)$, then for fixed g and any $w \in \mathcal{L}_P$, $\Pr[\text{Derive}(K, T, g, P) = w] = 1/N_P$. A real PRP gives computational indistinguishability under its security assumptions, not unconditional exact uniformity over keys.

Theorem 4 (context separation). Under HKDF and tweakable-PRP assumptions, distinct credential identities, salts, Root-Key generations, or policy epochs induce computationally separated derivation contexts. This is a computational claim; outputs from different generations in one permutation are sampling without replacement and are not statistically independent.

6. Credential lifecycle and migration

6.1. Version semantics

The dispatch pair ‘(derivationVersion, encoderVersion)’ is authenticated and fail-closed. Pair ‘(2,2)’ executes the existing CDR derivation and Encoder v2 without reinterpretation. Pair ‘(3,3)’ executes exact policy-space derivation.

Unknown pairs fail. A v2 record never becomes v3 merely because new software is installed.

Within a v3 credential lineage, the credential salt remains stable. A same-policy rotation keeps ‘policyEpoch’ and increments the epoch-local ‘credentialGeneration’. A policy change increments ‘policyEpoch’ and resets generation to zero; the policy hash in the tweak selects a different permutation. This separation is essential: generating a fresh salt at every rotation would select a new permutation and destroy Theorem 2’s sequence interpretation.

6.2. History exclusion across epochs

Independent policy epochs can have intersecting languages, so Theorem 2 does not prevent cross-epoch reuse. For a server history window h , the client retains authenticated descriptors of the recent committed versions and rederives their passwords; it does not store the values. Let E be the set of rederived recent passwords. The pending epoch tests successive unused generation inputs and selects the first output outside E .

Proposition 1 (bounded exclusion). Under a total permutation, if at least $|E \cap \mathcal{L}_P| + 1$ unused generation inputs remain, the process terminates after at most $|E \cap \mathcal{L}_P| + 1$ candidates. It fails when history cannot be authenticated or rederived, including history under an unavailable Root-Key generation. A configured history window above 64 is rejected.

The client-side check cannot predict undocumented server history rules. A server rejection remains a remote rotation outcome, not a reason to silently change the local generation.

6.3. Evidence-bounded remote rotation

Migration and rotation create a pending record before contacting the service. The remote adapter submits the candidate and attempts independent verification of both new and old passwords under a lockout budget and backoff policy. Four observations matter: ‘NewOnly’, ‘OldOnly’, ‘Both’, and ‘Neither’. Only authenticated evidence whose possible-state set is the singleton ‘NewOnly’ permits local commit. ‘OldOnly’ aborts; ‘Neither’ requires recovery; ‘Both’ is ambiguous for atomic replacement and may enter explicit old-password revocation only for an overlap-then-revoke adapter contract. Lost responses enter ‘UnknownOutcome’ and reconciliation rather than being interpreted as success.

Figure 1 shows the main path. An HTTP 200 alone is not commit evidence; an adapter must classify the response and verify authentication. The artifact

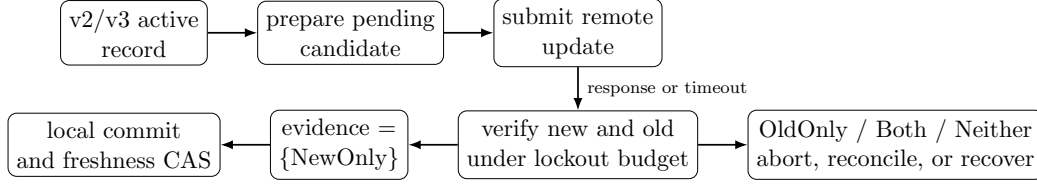


Figure 1: Evidence-bounded v3 migration and rotation. A positive submission response is not sufficient for local commit.

contains the state transition tests but does not yet provide two production Web adapters.

7. Recovery and freshness

The Root Key is random and split using standard Shamir 2-of-3 sharing into a managed-computer factor, USB factor, and offline or optional network recovery factor. A manifest identifies ‘vaultID’, Root generation, ‘shareSetID’, factor generations, and a key-confirmation value. Each share envelope is authenticated with a key derived only after a candidate Root Key is reconstructed. Recovery tries available share pairs, derives the candidate confirmation and share-MAC keys, compares the manifest KCV, and then verifies both envelopes. With one bad share among three, the client tests all three pairs; it does not claim to locate a malicious share from unauthenticated Shamir algebra alone.

The optional Network Recovery Profile distributes one encrypted network share as a fixed-size, erasure-coded object and gates release with a high-entropy capability, two prototype approvals, a 3-of-5 threshold-OPRF prototype, and a freshness claim. Approval binds the blinded request hash. Node handling is idempotent: the same operation nonce and request returns the same response, whereas the same nonce with a different request is rejected. Failure of this profile does not affect ‘managed computer + USB’ recovery.

This profile is supporting evidence, not the paper’s algorithmic contribution. It assumes honest-but-curious OPRF nodes and HMAC test approvers. Production use requires a reviewed threshold VOPRF, verifiable responses, DKG and proactive share refresh, public-key enterprise approvals, durable idempotency, a linearizable CAS freshness store, post-recovery share-set replacement, and real P2P adversarial evaluation.

8. Implementation

The Rust core separates ‘policy’, ‘derivation’, and lifecycle services. The policy compiler creates only reachable product states and stores all completion counts as ‘BigUint’. Public operations include exact count by length, total count, rank, unrank, and policy hash. Property tests compare enumeration and counting, exercise ‘Rank(Unrank(r))=r’, test values above 128 bits, and verify constraint families.

‘DomainPermutation’ isolates the mathematical construction from the FF1 research backend. The fixed vector binds all UUIDs, versions, salt, policy descriptor, Root Key, credential key, exact domain size, rank, and output password. Callers may cache a compiled policy; complete derivation otherwise compiles on demand. The migration service verifies the active CDR MAC, constructs a pending ‘(3,3)’ record, rederives the configured history window, selects a non-repeated generation, authenticates the pending CDR, and persists it without committing the old record.

The current implementation limitation is that an arbitrary finite-domain PRP backend with a proof-matched, total implementation has not replaced bounded FF1 cycle walking. Complex forbidden-substring compilation is also a simple suffix-state construction rather than a minimized Aho–Corasick product, which explains one stress-policy rejection.

9. Evaluation

All measurements are reproducible release-mode local baselines. They were run on one macOS x86-64 host. Raw JSON contains timestamps, sample counts, exact integers, and claim boundaries.

9.1. Compiler feasibility and exact spaces

The synthetic corpus contains 12 representative policies, not 12 measured enterprise deployments. Eleven compile within 250,000 states. Common class constraints require 9–254 states; edge and run constraints require 6,201–8,889 states; per-character totals reach 159,250 states. A stress case with long forbidden substrings over a 36-character alphabet is rejected at the state limit. Table 1 reports selected cases.

For the 18-character, 74-symbol case, the exact count is

$$N_P = 3923998958022941748259805567385600,$$

corresponding to 111.596 bits for a uniform variable over that language.

Table 1: Selected synthetic policy compilation results.

Policy	States	Compile (μ s)	$\log_2 N_P$
Lower/digit, length 8	9	999	41.36
Four classes, length 12	164	3,644	71.36
Four classes, length 18	254	6,972	111.60
No symbol at edges, length 16	7,531	170,006	95.63
Per-character total ≤ 2	159,250	721,724	34.91
Long forbidden-substring stress	rejected	386,308	—

Table 2: Distribution over 254 legal strings (100,000 samples).

Method	Empirical TVD	χ^2	max/min count
Encoder v2	0.06668	3011.61	2.91
Viable-edge uniform walk	0.02467	923.47	2.36
Whole-string rejection	0.02006	247.32	1.31
Uniform rank/unrank	0.02056	261.97	1.37

9.2. Distribution and collision experiments

The distribution policy contains all length-8 strings over $\{a, 0\}$ except the two single-class strings, hence $N_P = 254$. Each method generates 100,000 deterministic samples under fixed experimental seeds. Table 2 shows empirical distance to the uniform target. Whole-string rejection and uniform-rank generation have chi-square statistics close to their 253 degrees of freedom. Encoder v2 and the tested viable-edge-uniform walk depart more strongly. This result is implementation-specific; it does not show that every Xeger or DFA generator is biased.

Across all 254 generations in the toy domain, Encoder v2 produces 93 observed collisions and SHA-256 followed by truncation and modulo produces 87. An identity-bijection test oracle produces zero; it is not a cipher and isolates the structural property. The actual v3 FF1 backend produces zero observed collisions over 2,000 generations in the default large policy domain. The theoretical no-repeat claim follows from injectivity, not from these samples.

9.3. Local performance and recovery baseline

For the 18-character policy, 500 operations give rank and unrank medians near 1–2 microseconds. Complete v3 derivation, including HKDF, canonical

tweak, FF1 cycle walking, and unrank, has median 75.7 microseconds, P95 170.5 microseconds, and P99 266.5 microseconds. These values exclude storage, network, approval, adapter, and user delay.

The supporting peer-recovery baseline scans 100,000 fixed-size objects in approximately 934 ms, completes 1,000 local threshold releases at a mean of 558 microseconds, and reconstructs the same encrypted object for all ten 3-of-5 fragment combinations. Network RTT and human approval would dominate a real deployment, so these are cryptographic and memory-scan baselines only.

9.4. Lifecycle model checking

The bounded TLA+ model includes explicit migration, same-policy rotation, policy change, submission, unknown outcome, four remote observations, commit, and non-commit completion. With versions bounded to 4 and epochs/generations to 3, TLC 2.19 generates 127 states, finds 64 distinct states at depth 13, and reports no violation of type consistency, version-pair consistency, no silent v3 commit, committed-generation recording, salt stability, or ‘NewOnly’ commit evidence. The state count is small because cryptography, identities, payloads, and clocks are abstracted; it is not a proof of the entire system.

10. Security discussion

The principal unconditional properties are language compliance and non-repetition conditioned on successful execution of a permutation. Exact space size is a combinatorial fact. Uniformity and context separation require ideal-permutation or computational assumptions, respectively. Password outputs remain limited by N_P and by server-side online/offline attack conditions; deterministic generation cannot add entropy to a small policy language.

Root-Key compromise reveals every derivable credential in its generation. Credential salt is not a secret and prevents neither Root-Key compromise nor malicious endpoint derivation. Its role is domain separation and stable lineage identity. Changing it intentionally creates a new lineage and ends the within-permutation non-repeat scope.

Rollback protection is layered. CDR MACs, share-envelope MACs, KCV, and version binding detect tampering and mixed generations after key reconstruction. They cannot distinguish a complete, authentic old snapshot. That requires an independently administered durable freshness anchor. The

current persistent CAS service prototype must still be deployed and evaluated across failure domains before a rollback-resistance claim is justified.

Remote evidence is also bounded. Re-authentication can be affected by MFA, CAPTCHA, account lockout, caching, and authentication-replica lag. Adapters must declare response classification, verification method, lockout budget, and backoff. ‘Both’ is a first-class ambiguous state, not success.

11. Limitations and future work

The most important limitations are empirical and cryptographic. The policy corpus is synthetic; a defensible compatibility claim requires a documented corpus of real enterprise policies and a comparison with MFDPG on the same inputs. The compiler can suffer product-state explosion and currently lacks automaton minimization and optimized multi-pattern matching. Unicode normalization, dictionary or breached-password services, context-dependent remote rules, and unbounded semantic predicates are not supported.

The FF1 backend follows a draft minimum-domain rule and bounds cycle walking. A production instantiation should use a reviewed, proof-matched permutation whose implemented domain and failure semantics exactly match the security statement, or prove availability bounds acceptable to the application.

The evaluation lacks Windows and Linux measurements, two real password-change adapters, crash injection at every storage/network boundary, a deployed linearizable freshness service, and a user study of the offline recovery package. These are necessary before a systems-paper claim of production readiness. Network recovery additionally requires Byzantine-verifiable threshold release, signed independent approvals, proactive refresh, durable idempotency, real transport, and post-recovery revocation.

A separate theoretical question remains open for this project: dynamic, intersecting policy languages under adversarial policy changes and a bounded history state. Before treating that as a new mathematical contribution, it requires an additional prior-art audit across dynamic FPE, constrained sampling without replacement, and state lower bounds.

12. Conclusion

Exact policy-space derivation can replace an ad hoc composition encoder without inventing new cryptographic primitives. The construction compiles a bounded policy, counts and bijectively indexes every accepted password,

and maps credential generations through a domain-separated finite-domain permutation. Its useful contribution is the lifecycle around this known rank-then-encipher idea: explicit version dispatch, stable lineage salt, policy epochs, history-window rederivation, remote-evidence reconciliation, and recoverable Root-Key generations. The prototype and experiments show feasibility and make state explosion, backend totality, and deployment gaps measurable. Those boundaries are central to the claim: the work is an evidence-backed research prototype for legacy password systems, not a new FPE or a finished production recovery platform.

Artifact availability

The artifact contains the Rust implementation, fixed derivation vector, synthetic policy corpus generator, raw JSON measurements, TLA+ model, and reproduction commands. An anonymized repository URL will be added for review.

References

- [1] B. Ross, C. Jackson, N. Miyake, D. Boneh, J. C. Mitchell, Stronger password authentication using browser extensions, in: 14th USENIX Security Symposium, 2005.
- [2] M. Horsch, A. Hulsing, J. Buchmann, Palpas—passwordless password synchronization, in: 10th International Conference on Availability, Reliability and Security, 2015, pp. 30–39. doi:10.1109/ARES.2015.23.
- [3] F. A. Maqbali, C. J. Mitchell, Autopass: An automatic password generator, arXiv:1703.01959 (2017).
URL <https://arxiv.org/abs/1703.01959>
- [4] M. Billefont, Spectre: A password cipher (2023).
URL <https://spectre.app/>
- [5] V. Nair, D. Song, MFDPG: Multi-factor authenticated password management with zero stored secrets, in: Network and Distributed System Security Symposium, 2024. doi:10.14722/ndss.2024.24646.
- [6] A. V. Goldberg, M. Sipser, Compression and ranking, in: 17th Annual ACM Symposium on Theory of Computing, 1985, pp. 440–448.

- [7] J. Black, P. Rogaway, Ciphers with arbitrary finite domains, in: Topics in Cryptology—CT-RSA 2002, 2002, pp. 114–130.
- [8] M. Bellare, T. Ristenpart, P. Rogaway, T. Stegers, Format-preserving encryption, in: Selected Areas in Cryptography, 2009, pp. 295–312.
URL <https://eprint.iacr.org/2009/251>
- [9] J.-B. Oudinet, A. Denise, A. Genitrini, A new dichotomic algorithm for the uniform random generation of words in regular languages, Theoretical Computer Science 502 (2013) 165–176. doi:10.1016/j.tcs.2012.07.025.
- [10] National Institute of Standards and Technology, Methods for format-preserving encryption: SP 800-38G revision 1, second public draft, Tech. rep., NIST (2025).
URL <https://csrc.nist.gov/pubs/sp/800/38/g/r1/2pd>
- [11] F. B. Durak, S. Vaudenay, Breaking the FF3 format-preserving encryption standard over small domains, in: Advances in Cryptology—CRYPTO 2017, 2017.
- [12] V. Nair, D. Song, Multi-factor key derivation function (MFKDF) for fast, flexible, secure, and practical key management, in: 32nd USENIX Security Symposium, 2023, pp. 2097–2114.
- [13] M. Scarlata, M. Backendal, M. Haller, MFKDF: Multiple factors knocked down flat, in: 33rd USENIX Security Symposium, 2024, pp. 4301–4318.
URL <https://www.usenix.org/conference/usenixsecurity24/presentation/scarlata>
- [14] A. Shamir, How to share a secret, Communications of the ACM 22 (11) (1979) 612–613.